第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

——以 DVWA SQL Injection 模块为例

1.页面中的输入为什么会与数据库查询有关?

以 DVWA 中的“查找用户 ID”功能为例：
页面有一个输入框，让用户输入 User ID，点击提交后，页面会显示该用户的姓名等信息。

这些用户数据并不是写死在页面里的，而是存储在 MySQL 数据库 中。
用户输入不同的 ID，页面就需要去数据库里查找对应的记录。
因此：页面输入的本质是告诉数据库“你要查什么”，没有输入，查询就没有条件。

2. 一个最小业务链路是什么?

输入条件 → 数据库查询 → 返回结果

以 **DVWA low** 级别为例：

1）输入条件：用户在页面的 User ID 文本框中输入一个数字例如：1），然后点击 Submit 按钮提交。

2)数据库查询：前端将用户输入的 1 通过 HTTP 请求发送给后端服务器。后端（PHP 脚本）收到后，直接将用户输入拼接到 SQL 查询语句中，然后向数据库执行查询。

3) 返回结果：

- **查询到记录：**数据库返回 user_id 为 1 的用户信息（如 First name: admin，Surname: admin），服务器将其呈现在网页上。

- **未查询到记录：**如果输入一个不存在的 ID（如 99），数据库返回空，页面无结果或显示空内容。

这就是一个完整的最小业务链路。

3. 用户输入是如何进入查询条件的?

在 DVWA 的这个业务场景中，**用户输入的 User ID 值**通过以下路径进入查询条件：

- 用户在页面输入 1

- 浏览器通过 HTTP GET 请求将 1 发送给后端
（例如：http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit）

- 后端 PHP 代码接收参数：
  $id = $_GET[‘id’]

- 将该参数直接拼接到 SQL 语句中：
  $sql = "SELECT first_name, last_name FROM users WHERE user_id = $id"

- 数据库执行这条包含用户输入的 SQL

结论：用户输入 → 后端接收 → 拼接到 SQL 条件中 → 执行查询

4. 为什么这为 SQL 注入创造了可能性？

因为后端程序直接把用户输入拼接到 SQL 语句中，没有做任何检查和过滤。

举例说明：
1)正常用户输入 1，SQL 为：
SELECT first_name, last_name FROM users WHERE user_id = 1

2)恶意用户输入 1 OR 1=1，SQL 变为：
SELECT first_name, last_name FROM users WHERE user_id = 1 OR 1=1

由于 1=1 永远为真，这条 SQL 会返回表中所有用户的姓名，造成数据泄露。

更危险的输入如 1; DROP TABLE users; --，可能直接删除整张用户表。

核心原因：
用户输入被当作 SQL 代码 执行，而不仅仅是 数据。

任务 2：DVWA SQL Injection Low 现象记录

1.实验页面或实验目标

- **页面**：DVWA → SQL Injection (Low)
- **目标**：输入 User ID，返回对应的 First name / Last name

2. 普通输入与异常输入的对比

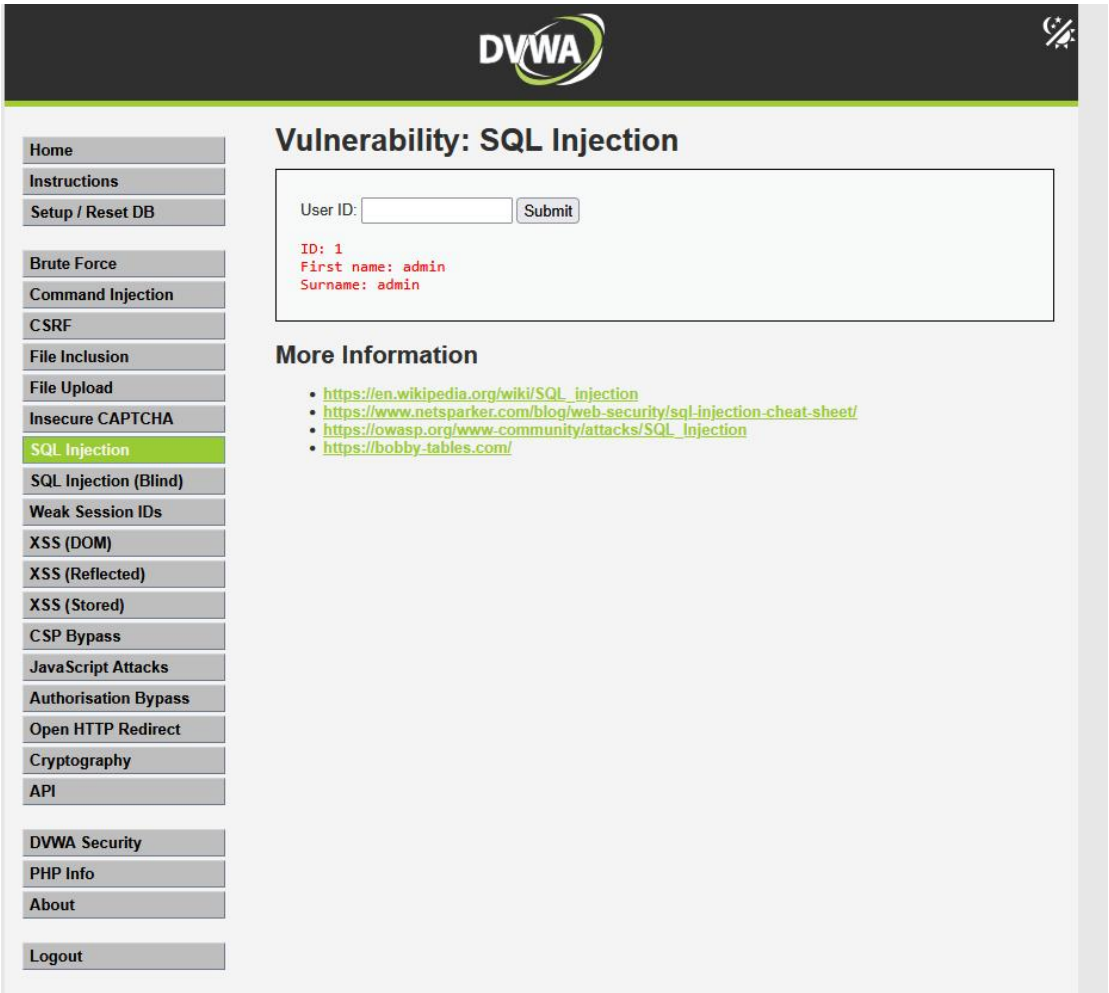
普通输入： 1

页面返回： ID: 1

First name: admin

Surname: admin

逻辑作用： 作为 ID 值查询



(输入 1，返回 admin 单条记录)

异常输入：1’OR ’1’=’1

返回结果：

ID: 1

First name: admin

Surname: admin

ID: 2

First name: gordon

Surname: brown

ID: 3

First name: pablo

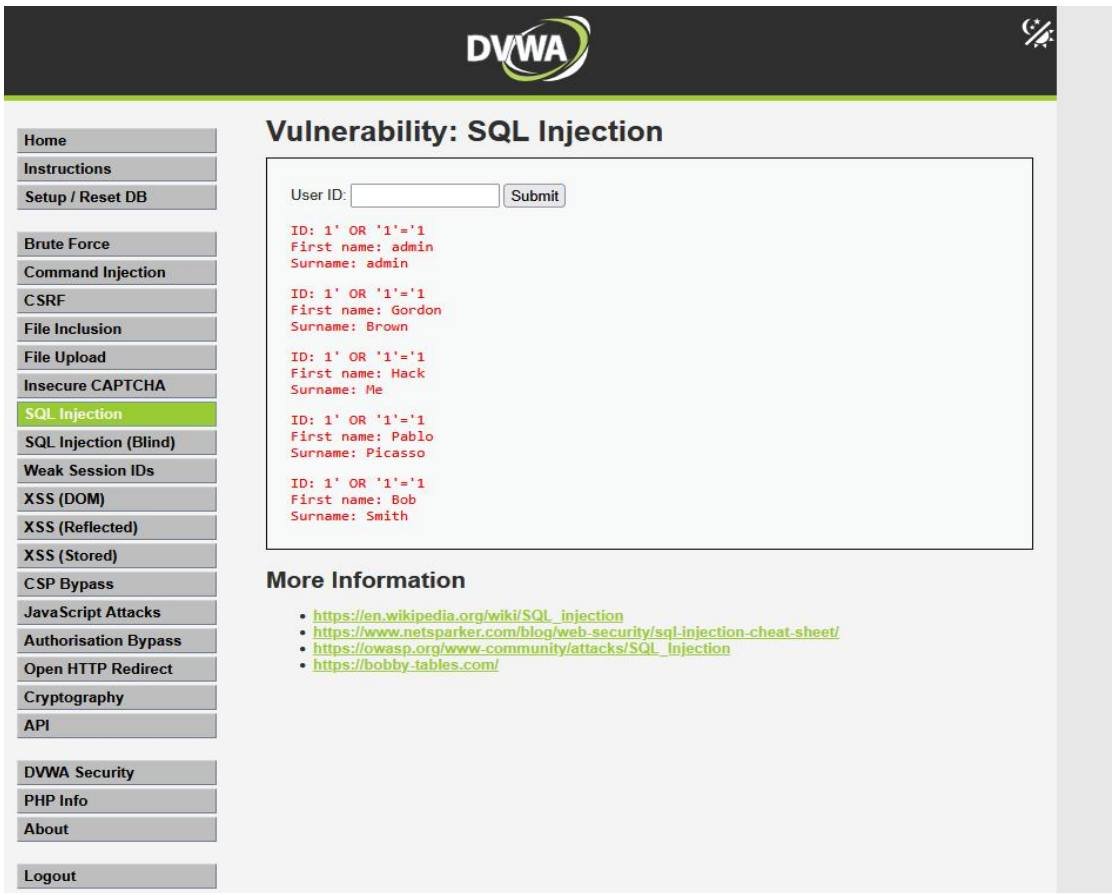
Surname: picasso

ID: 4

First name: smith

Surname: smith

逻辑作用：改变 WHERE 条件逻辑



(输入 1’OR ’1’=’1，返回 admin 多条记录)

3. 返回结果的差异

- **输入 1：**返回 1 条 记录（admin）
- **输入 1’OR ’1’=’1：**返回 多条 记录（admin, gordon, pablo, smith 等全部用户）

4. 为什么这种差异不像“正常查询变化”

正常查询变化是：输入 1 → 查 admin；输入 2 → 查 gordon。

差异始终是单条记录;而输入 1’OR ’1’=’1 使 WHERE 条件恒为真，返回了全表数据。

这个差异支撑的判断是：
用户输入被拼接到了 SQL 的逻辑结构中，而不是仅仅作为一个参数值。这是 SQL 注入的直接证据，而非业务上的正常查询变化。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

输入 → 拼接 → 解析 → 执行 → 回显或行为差异

2. 为什么根因是“结构被污染”而不是“字符本身危险”？

回答:危险的不是 1 或 OR 或 = 这些字符，而是输入内容进入了 SQL 语法结构。

(例如在参数化查询中，输入 1 OR 1=1 只会被当作字符串值查询，不会改变逻辑。)

结论：问题发生在“输入被拼接到查询结构”，而非字符本身。

3. Low 与 Medium 的差异在哪里?

- **Low：**直接拼接，无任何防护。
- **Medium：**使用 mysqli_real_escape_string() 对输入转义，并尝试用 intval() 限制类型。但仍有绕过方式（如构造数字+字符串组合）。
- **核心差异：**Medium 是输入过滤/转义思路，仍未做到结构分离。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里?

- 普通注入：页面直接回显数据（如用户名、错误信息），攻击者可以直接看到查询结果。
- 盲注：页面不直接回显数据，攻击者通过 true/false 差异（布尔盲注）或时间延迟（时间盲注）推断数据。

- 关键差异：证据形式不同（一个直接回显，一个间接推断）。

5. 为什么盲注和普通注入根因相同，但证据形式不同？

1）**根因相同：**两者都是因为用户输入被拼接到 SQL 查询结构中，导致查询逻辑被污染。

2）证据形式不同的原因在于应用程序的响应方式：

- 如果应用程序将查询结果直接输出到页面上 → 普通注入（有回显）
- 如果应用程序只根据查询是否返回结果来改变页面行为（如显示“存在”/“不存在”） → 布尔盲注
- 如果应用程序没有任何可见变化，但数据库可以执行延时函数 → 时间盲注

结论：根因相同（结构被污染），只是“证据的呈现方式”不同。

任务 4：SQL 注入防护清单与解释

1. 参数化查询

1）它解决什么问题？

将 SQL 代码结构（如 SELECT ... WHERE user_id = ?）与用户输入的数据分离。数据库先解析 SQL 结构，再把用户输入当作纯数据绑定进去。即使用户输入 1 OR 1=1，它也只是一个普通字符串，不会改变查询逻辑。

2）它控制哪一层？

根因控制层。直接解决“查询结构被污染”这个根本原因，而不是在后果上修补。

3）它不能替代什么？

不能替代输入校验。参数化查询虽然能防止注入，但不阻止用户输入超长字符串或非法字符（如<script>），这些可能在其他环节（如 XSS、存储溢出）造成问题。

（**DVWA 对应示例：**

Impossible 级别使用了$mysqli->prepare()，这是参数化查询的典型实现。）

2. 输入校验

1）**它解决什么问题？**

确保输入数据符合预期格式。例如：User ID 应该是数字，则检查

is_numeric()；用户名应该只包含字母数字，则用正则过滤。它可以提前拒绝明显非法的输入，减少攻击面。

2）它控制哪一层？

暴露面控制层（也称深度防御/外围防护）。在用户输入到达查询构造之前进行筛选，减少恶意输入进入核心逻辑的机会。

3）它不能替代什么？

- 不能替代参数化查询。因为：
- 校验逻辑可能被绕过（如 Unicode 欺骗、双重编码）
- 校验规则可能不完整（如忘记过滤某个特殊字符）
- 某些场景（如搜索关键词）很难做到严格白名单校验

（**DVWA 对应示例：**

Medium 级别使用 intval()尝试将输入转为整数，这是一种输入校验尝试，但不够彻底，仍可被绕过。）

3. 错误处理

1）它解决什么问题？

防止数据库错误信息（如表名、字段名、SQL 语法）直接暴露给用户。攻击者常利用错误信息推断数据库结构和注入是否成功。自定义错误页面或记录日志到服务端，可以有效隐藏这些信息。

2）它控制哪一层？

后果范围控制层。它不阻止注入发生，但可以减少攻击者获得的信息

量，增大攻击难度。

3）它不能替代什么？

不能替代参数化查询或输入校验。即使不显示错误信息，注入仍然可能成功（如盲注情况）。错误处理只是让攻击者“看不见”，而不是“不能执行”。

（**DVWA 对应示例：**

在 Low 级别中，SQL 执行错误会直接显示在页面上（如 You have an error in your SQL syntax...）。Impossible 级别通过 die()和自定义错误处理避免了敏感信息泄露。）

4. 最小权限

1）它解决什么问题？

限制数据库账户的权限。例如：Web 应用只应该用只读账户查询数据，不应使用 root 或拥有 DROP、INSERT 权限的账户。这样即使 SQL 注入成功，攻击者也无法删除表或篡改数据。

2）它控制哪一层？

后果范围控制层。这是“防御纵深”的最后一环，限制漏洞被利用后的最大损失。

3)它不能替代什么？

- 不能替代参数化查询。因为：

- 只读账户仍可能被用于越权读取其他表的数据
- 某些业务必须的写操作（如留言板、订单提交）无法用只读账户
- 最小权限是降低风险，而不是消除漏洞

(DVWA 对应示例：

DVWA 默认使用一个具有完整权限的数据库用户。理论上生产环境应单独创建一个只对 users 表有 SELECT 权限的账户用于查询操作。)

5. 可补充其他合理措施

补充措施：使用存储过程（Stored Procedures）

1)它解决什么问题？

将 SQL 逻辑封装在数据库端，通过参数调用。如果合理使用（结合参数化），可以减少动态 SQL 拼接的机会。

2)它控制哪一层？

根因控制层（辅助）。但它本身不安全：如果在存储过程内部仍然拼接输入，则依然存在注入风险。

3)它不能替代什么？

不能替代参数化查询。存储过程只有与参数化结合使用时才是安全的。

防护措施分层总结图（逻辑关系）

用户输入

↓

【暴露面控制】← 输入校验（提前拦截明显非法输入）

↓

【根因控制】 ← 参数化查询（核心：结构分离）★最关键

↓

数据库执行

↓

【后果范围控制】← 错误处理（不泄露信息）

↓

【后果范围控制】← 最小权限（限制损失上限）

收口项 1：SQL 注入因果链/防护关系图

环节	因果链	防护控制点
输入点	用户输入特殊构造的字符串	输入校验（暴露面控制）
数据库解析执行	输入改变了 SQL 语法结构	（无直接防护，由参数化预防）
结果/行为差异	回显全部数据 / 延迟 / 报错	错误处理（后果范围控制）
查询构造	输入被直接拼接到 SQL 中	参数化查询（根因控制）
最终影响	数据泄露、篡改、删除	最小权限（后果范围控制）

因果主链：

输入点 → 拼接 → 解析/执行 → 差异证据 → 根因（结构污染）

收口项 2：学生自评表（2 分）

- 我本章最清楚的一点是什么？

SQL 注入的根因是“结构被污染”，而非某个特殊字符。防护的核心是参数化查询。

- 我最容易混淆的一点是什么？

之前容易把“普通注入有回显、盲注无回显”误认为根因不同，现在已经分清是证据形式不同。

- 我本次作业中哪一段最需要教师复核？

任务 4 中“各措施控制哪一层”的分类，特别是输入校验算暴露面还是根因辅助。

- 如果重做一次，我会优先改哪一部分？

关系图会画成更直观的流程图，而不是表格。

收口项 3：AI 输出审核与人工复核记录

- 哪些内容使用过 AI 辅助：

任务 1～4 的框架结构、任务 3 判断题的解释表述、收口项 1 的关系表参考。

- 哪些判断是自己复核后保留的：

所有 DVWA 实验现象描述、Low 与 Medium 差异分析、盲注与普通注入的证据形式区分。

- 哪些内容发现 AI 输出不准确并进行了修正：

AI 最初将 Blind SQL Injection 描述为“根因不同”，已修正为“根因相同，证据形式不同”。

AI 曾将输入校验放在根因控制层，已修正为暴露面控制。

- 最终如何确认结论和证据是一一对应的：

每个结论旁都标注了对应的 DVWA 实验现象（如：输入 1 OR 1=1 返回全部数据 → 证明结构被污染），并做了本地复现验证。